EY DevOps Interview

Complete Q&A; Prep Guide — Sanket Chopade

ROUND 1

Kubernetes

Q: What is a Kubernetes namespace?

A namespace is a logical boundary within a cluster that isolates resources. It's used to separate environments (dev/prod), teams, or projects. Resources in one namespace don't clash with another. Default namespaces: default, kube-system, kube-public.

[Isolation | Multi-tenancy | Resource Quotas]

Q: Steps to deploy an image in a Kubernetes cluster?

1. Write app code and Dockerfile → 2. Build image: docker build → 3. Push to registry (ECR/DockerHub) → 4. Write Deployment YAML with image reference → 5. Write Service YAML to expose it → 6. kubectl apply -f deployment.yaml → 7. Verify: kubectl get pods, kubectl describe pod

[Deployment | Service | kubectl apply]

Q: Worker node went down. What are your next steps?

1. kubectl get nodes — check status (NotReady) → 2. kubectl describe node — check events/conditions → 3. SSH into the node, check kubelet: systemctl status kubelet → 4. Restart kubelet if needed: systemctl restart kubelet → 5. kubectl drain — safely evict pods to other nodes → 6. kubectl cordon — mark unschedulable → 7. Fix node or replace it, then kubectl uncordon.

[kubelet | drain | cordon | describe]

Q: How to connect a pod to another pod in a different namespace?

Use the fully-qualified domain name (FQDN): ..svc.cluster.local. Example: if Service 'payment-svc' is in namespace 'billing', connect via: http://payment-svc.billing.svc.cluster.local:8080. Also ensure NetworkPolicies allow cross-namespace traffic.

[FQDN | Cross-namespace | NetworkPolicy]

Q: What is Persistent Volume (PV) and Persistent Volume Claim (PVC)?

PV is a piece of storage provisioned by an admin (NFS, EBS, etc.) — it exists independently of pods. PVC is a request for storage by a pod — it binds to a matching PV. Flow: StorageClass → PV created (static or dynamic) → PVC requests it → Pod mounts PVC as volume. This decouples storage from pod lifecycle.

[PV = Storage Resource | PVC = Storage Request | StorageClass]

Q: How to handle Role-Based Access Control (RBAC) in Kubernetes?

RBAC has 4 objects: 1) Role — defines permissions within a namespace, 2) ClusterRole — cluster-wide permissions, 3) RoleBinding — binds Role to a user/ServiceAccount in a namespace, 4) ClusterRoleBinding — binds ClusterRole cluster-wide. Example: create a Role with get/list on pods, then RoleBinding to a ServiceAccount used by your app.

[Role | ClusterRole | RoleBinding | ServiceAccount]

Q: What is Helm?

Helm is the package manager for Kubernetes. It bundles K8s manifests into 'charts'. Key concepts: Chart (package), values.yaml (configurable inputs), Release (deployed instance of a chart), Repository (chart storage). Commands: helm install, helm upgrade, helm rollback. It enables templating and version management of K8s apps.

[Charts | values.yaml | helm install/upgrade/rollback]

Jenkins / CI-CD

Q: Explain CI/CD stages in detail.

Typical pipeline: 1. Checkout — git clone from SCM → 2. Build — mvn/gradle/npm build → 3. Unit Test — run automated tests → 4. SAST — SonarQube code quality scan → 5. Docker Build — docker build -t image:tag → 6. Docker Push — push to ECR/DockerHub → 7. Deploy to Dev/QA — kubectl apply or helm upgrade → 8. Integration/E2E Tests → 9. Approval Gate (for prod) → 10. Deploy to Prod → 11. Notify — Slack/email notification.

[Checkout | Build | Test | SAST | Docker | Deploy | Notify]

Terraform

Q: What is a resource block in Terraform?

A resource block defines a single infrastructure object. Syntax: resource "" "" { ... }. Example: resource "aws_instance" "web" { ami = "ami-123" instance_type = "t3.micro" }. The two labels are the resource type and the local name used to reference it elsewhere.

[resource | provider_type | local_name]

Q: What is a state file in Terraform?

terraform.tfstate is a JSON file that maps your config to real-world resources. It tracks what Terraform manages, resource IDs, metadata, and dependencies. Without it, Terraform can't know what already exists. In teams, store remotely in S3 + DynamoDB for locking to prevent concurrent modifications.

[terraform.tfstate | Remote Backend | S3 + DynamoDB lock]

Q: Deploy Linux/Windows from a single Terraform script?

Use variables and conditionals. Define a variable for OS type, then use count or for_each with conditional AMI selection: ami = var.os_type == "linux" ? var.linux_ami : var.windows_ami. Similarly, user_data or WinRM config changes based on the OS. This makes the script reusable for both.

[Variables | Conditional expressions | ami selection]

Q: Terraform script to provision S3 with/without versioning?

Use a variable: variable "enable_versioning" { default = true }. Then in the resource: versioning { enabled = var.enable_versioning }. The user passes -var="enable_versioning=false" at runtime to disable it. This makes versioning a configurable toggle.

[versioning block | variable | -var flag]

Q: How does Terraform resolve dependencies?

Implicit dependency: when resource B references resource A's attribute (e.g., aws_subnet.id), Terraform auto-detects the dependency and creates A first. Explicit dependency: use depends_on = [aws_vpc.main] when there's no direct reference but ordering still matters. Terraform builds a dependency graph (DAG) and executes in the correct order.

[Implicit | Explicit depends_on | DAG]

Q: Handle dependencies across separate modules (VPC, EC2)?

Pass outputs of one module as inputs to another. In root main.tf: call vpc module first, then pass module.vpc.subnet_id as input to the ec2 module. Terraform resolves this automatically — ec2 module depends on vpc module's output, so vpc is provisioned first. This is the standard pattern for modular IaC.

[Module outputs | Module inputs | Cross-module dependencies]

Linux

Q: How to securely SSH to a VM?

Best practices: 1. Use SSH key pairs (not passwords) — ssh-keygen, copy public key to ~/.ssh/authorized_keys → 2. Disable password auth in /etc/ssh/sshd_config: PasswordAuthentication no → 3. Use specific port instead of 22 (optional) → 4. Use Security Groups / firewall rules to allow SSH only from trusted IPs → 5. Use bastion host / jump server for accessing private instances → 6. Use AWS Systems Manager Session Manager for passwordless access (no SSH port needed).

[Key-based auth | Security Groups | Bastion Host | SSM]

ROUND 2

Docker

Q: What is ENTRYPOINT vs CMD? Which can be overridden?

CMD sets the default command and CAN be overridden at runtime (docker run myimage /bin/bash). ENTRYPOINT sets the fixed executable that always runs — it CANNOT be overridden without --entrypoint flag. Best practice: use ENTRYPOINT for the main binary (e.g., ENTRYPOINT ["python"]) and CMD for default arguments (CMD ["app.py"]). Together they form: python app.py.

[CMD = overridable | ENTRYPOINT = fixed executable]

Q: What are layers in container images?

Every instruction in a Dockerfile creates a read-only layer. FROM creates the base layer, RUN/COPY/ADD create additional layers. Layers are cached — if a layer hasn't changed, Docker reuses the cache. This makes builds faster. Layers are stacked and shared across images, saving disk space.

[Read-only | Cached | Stacked | Shared]

Q: Difference between image layer and container layer?

Image layers are read-only and shared across all containers using that image. When a container starts, Docker adds a thin writable layer on top (called the Container Layer or Copy-on-Write layer). Any writes/changes happen only in this writable layer — the underlying image layers remain unchanged. When the container is deleted, this writable layer is lost (unless you use volumes).

[Image = Read-only layers | Container = adds writable CoW layer]

Q: Implications of running a container with root-level permission?

Security risks: 1. Container escape — a compromised process can potentially access the host OS → 2. Host filesystem access — root in container can mount/read host paths → 3. Privilege escalation — can interact with host kernel → 4. No isolation benefit. Best practice: use USER instruction in Dockerfile to run as non-root, use --security-opt=no-new-privileges, use read-only filesystems where possible.

[Container escape | Host access | Privilege escalation | USER instruction]

Kubernetes — Deep Dive

Q: HPA vs VPA — What are they?

HPA (Horizontal Pod Autoscaler): scales the number of pod replicas based on CPU/memory or custom metrics. VPA (Vertical Pod Autoscaler): adjusts the CPU/memory requests and limits of existing pods. Use HPA for stateless apps that can scale out. Use VPA when you need right-sized resource allocation. They can be used together but need care to avoid conflicts.

[HPA = more pods | VPA = bigger pods | autoscaling/v2]

Q: What is the API Server and its use case?

The kube-apiserver is the central control plane component. All kubectl commands, controller actions, and scheduler decisions go through it. It validates and processes REST requests, persists state in etcd, and is the only component that talks directly to etcd. Use cases: authentication, authorization (RBAC), admission control, and serving the K8s API.

[Control plane hub | etcd gateway | kubectl endpoint | RBAC enforcement]

Q: How does the Kubernetes Scheduler work?

The scheduler watches for unscheduled pods (pods with no nodeName set). It runs two phases: 1. Filtering — eliminates nodes that don't meet requirements (resources, taints, nodeSelector, affinity) → 2. Scoring — ranks remaining nodes (least loaded, affinity rules) → 3. Binding — assigns the highest-scored node to the pod by setting nodeName. This is updated via the API server.

[Filter → Score → Bind | Affinity | Taints | Resources]

Q: How to direct external traffic to pods? (non-trivial answer)

Full flow: External client → DNS (Route53) → Load Balancer (AWS ALB/NLB) → Ingress Controller (nginx/AWS LBC) → Ingress resource (routing rules by path/host) → ClusterIP Service → Pod. The Ingress Controller handles TLS termination, path-based routing, and rewrites. This is more powerful than a simple NodePort or LoadBalancer Service because it handles multiple apps via one LB.

[DNS → ALB → Ingress Controller → Ingress → Service → Pod]

Q: How to handle secrets in Kubernetes + AWS Secrets Manager?

Option 1 — External Secrets Operator: Install ESO, create an ExternalSecret resource that pulls from AWS Secrets Manager and syncs to a K8s Secret. Use IRSA (IAM Roles for Service Accounts) to give the ESO pod permission to read secrets. Option 2 — CSI Secrets Store Driver: Mount secrets directly as files/env vars into pods via a SecretProviderClass. Both require IRSA for AWS auth.

[External Secrets Operator | CSI Driver | IRSA | SecretProviderClass]

Q: Taints and Tolerations?

Taints are applied to nodes to repel pods: kubectl taint nodes node1 key=value:NoSchedule. Pods must have a matching Toleration to be scheduled on that node. Use cases: dedicated nodes for GPU workloads, reserving nodes for specific teams, or isolating critical workloads. Node Affinity is used to attract pods; Taints/Tolerations are used to repel them.

[Taint on Node | Toleration on Pod | NoSchedule | NoExecute]

Q: Labels vs Selectors?

Labels are key-value metadata attached to K8s objects (pods, nodes, services): app=frontend, env=prod. Selectors are used to filter/query objects by labels. Services use selector to route traffic to matching pods. ReplicaSets use selector to manage their pods. Selectors are the glue between K8s resources.

[Labels = metadata | Selectors = filter/link resources]

Q: End-to-end traffic flow in K8s (website to pod, including TLS)?

1. User browser → DNS resolves to ALB IP → 2. ALB receives HTTPS request → 3. TLS termination at ALB or Ingress Controller (cert from ACM or cert-manager) → 4. Ingress Controller reads Ingress rules → routes to ClusterIP Service → 5. Service selects matching pods via label selector → 6. kube-proxy / iptables routes to pod IP → 7. App processes request and responds in reverse path.

[DNS → ALB → TLS Termination → Ingress → Service → Pod]

Q: What is TLS? How to generate, store, and rotate certificates?

TLS (Transport Layer Security) encrypts data in transit between client and server, preventing eavesdropping and MITM attacks. Generate: use cert-manager (in-cluster) with Let's Encrypt as ClusterIssuer — it auto-issues certificates. Store: as Kubernetes Secret of type kubernetes.io/tls with tls.crt and tls.key. Rotate: cert-manager auto-renews before expiry. For AWS ACM, certificates are auto-rotated by AWS.

[cert-manager | Let's Encrypt | ClusterIssuer | kubernetes.io/tls Secret]

Q: What is CNI (Container Network Interface)?

CNI is a standard interface for configuring networking in containers. It defines how pods get IP addresses and communicate. K8s uses CNI plugins for pod networking. Common plugins: AWS VPC CNI (assigns VPC IPs to pods), Calico (network policies), Flannel (simple overlay), Cilium (eBPF-based). Without CNI, pods can't communicate.

[AWS VPC CNI | Calico | Cilium | Pod IP assignment]

AWS

Q: How to give Lambda permission to write to S3?

Create an IAM Role for Lambda with an inline/managed policy granting s3:PutObject on the target bucket ARN. Attach this role to the Lambda function as its Execution Role. The exact role needed: AWSLambdaBasicExecutionRole (for CloudWatch logs) + a custom policy for S3 write. Never hardcode credentials in Lambda code.

[IAM Execution Role | s3:PutObject | AWSLambdaBasicExecutionRole]

Q: Lambda ZIP size exceeded limits — best practices?

Lambda limits: 50MB zipped, 250MB unzipped. Solutions: 1. Lambda Layers — move shared dependencies (libraries, runtimes) to a Layer, reuse across functions → 2. Container Image deployment — up to 10GB image stored in ECR → 3. Store large assets in S3 and load at runtime → 4. Use slim base images, remove unused packages → 5. Split large function into smaller focused functions.

[Lambda Layers | Container Image (10GB) | S3 assets | ECR]

Q: Install dependencies on EC2 at deploy time?

Option 1 — User Data script: add a bash script in EC2 launch configuration that runs at first boot (yum/apt install, pip install). Option 2 — AWS Systems Manager (SSM) Run Command: remotely execute scripts on running instances without SSH. Option 3 — Bake dependencies into an AMI using Packer. Option 4 — Use Ansible playbooks for idempotent configuration management.

[User Data | SSM Run Command | AMI baking | Ansible]

Q: S3 event notifications → Lambda + EventBridge?

S3 can trigger Lambda directly on events (s3:ObjectCreated:Put, s3:ObjectCreated:Get). Configure in S3 bucket → Properties → Event Notifications → Add Lambda function. For advanced routing: use EventBridge — enable S3 event delivery to EventBridge, then create EventBridge rules to route to Lambda, SQS, SNS based on event patterns. EventBridge is more flexible for complex routing.

[S3 Event Notifications | EventBridge rules | Lambda trigger]

Terraform — Advanced

Q: State file locking — no DynamoDB needed now?

As of Terraform v1.6+, S3 backend supports native state locking without DynamoDB using the use_lockfile = true option. Previously you needed a DynamoDB table to prevent concurrent state modifications. The new S3 native locking writes a .tflock file in the same S3 bucket. This simplifies setup significantly.

[Terraform v1.6+ | S3 native locking | use_lockfile = true]

Q: Managing state files for 20 projects x 4 environments?

Best approach: separate S3 backend per environment with naming convention. Use a key pattern like: key = "projects///terraform.tfstate". All in one bucket with IAM policies per team. Alternative: Terraform Workspaces (simpler but shares backend config). For enterprise: HCP Terraform (Terraform Cloud) with Projects and Workspaces providing full isolation, RBAC, and audit logs.

[S3 backend key namespacing | Workspaces | HCP Terraform]

Q: What is a dynamic block in Terraform?

Dynamic blocks allow you to generate repeated nested blocks programmatically. Example: instead of writing multiple ingress blocks manually in a security group, use dynamic "ingress" { for_each = var.ingress_rules content { from_port = ingress.value.from_port ... } }. This reduces repetition and makes config data-driven.

[dynamic | for_each | content block | DRY principle]

Q: What is for_each/map() in Terraform?

for_each is used to create multiple resource instances from a map or set. Example: for_each = var.buckets creates one S3 bucket per map entry, with each.key and each.value available. The map() function constructs a map from key-value pairs. Together they enable data-driven, DRY Terraform configs instead of duplicating resource blocks.

[for_each | each.key / each.value | map() | DRY]

Q: How to limit deployments to 5 regions using Terraform Enterprise?

Use HashiCorp Sentinel — a policy-as-code framework for Terraform Enterprise/HCP Terraform. Write a Sentinel policy that checks the aws_region variable and enforces it must be within an allowed list of 5 regions. If the policy fails, the apply is blocked. This is enforced at the organizational level before any infrastructure changes happen.

[Sentinel | Policy-as-code | Terraform Enterprise | allowed regions]

DevSecOps

Q: SAST vs DAST?

SAST (Static Application Security Testing): analyzes source code without executing it. Run early in CI (SonarQube, Checkmarx). Finds: SQL injection patterns, hardcoded secrets, code quality issues. DAST (Dynamic Application Security Testing): tests the running application by sending requests. Tools: OWASP ZAP, Burp Suite. Finds: XSS, authentication flaws, runtime vulnerabilities. SAST = shift left; DAST = runtime validation.

[SAST = code analysis | DAST = runtime testing | SonarQube | OWASP ZAP]

Q: SonarQube stage in Jenkins pipeline?

In Jenkinsfile: stage('SAST') { steps { withSonarQubeEnv('SonarQube') { sh 'mvn sonar:sonar -Dsonar.projectKey=myapp' } } }. Then add a Quality Gate stage that waits for SonarQube analysis and fails the pipeline if gate fails: waitForQualityGate(abortPipeline: true). For .NET: use dotnet sonarscanner. For Java: use maven/gradle sonar plugin.

[withSonarQubeEnv | waitForQualityGate | Quality Gate | abortPipeline]

CI/CD — Jenkins Plugins

Q: Common Jenkins plugins used in DevOps pipelines?

Key plugins: 1. Git Plugin — SCM checkout → 2. Pipeline / Blue Ocean — pipeline as code → 3. Docker Pipeline — docker.build, docker.push in pipeline → 4. Kubernetes CLI / Kubernetes Plugin — kubectl commands, dynamic agents → 5. SonarQube Scanner — SAST integration → 6. AWS Steps / Amazon ECR — push to ECR → 7. Credentials Binding — securely inject secrets → 8. Slack Notification — send build alerts → 9. Email Extension — email notifications → 10. Build Timestamp — versioning.

[Git | Docker Pipeline | SonarQube | Kubernetes | Credentials Binding | Slack]